SDS/Xerox Sigma 7 CPU Design

Overview

The Sigma 7 is a 32-bit computer designed by Scientific Data Systems (SDS), later Xerox Data Systems. It uses **custom Diode-Transistor Logic (DTL)** throughout. The CPU employs a **hardwired control unit** with a fixed phase-based timing structure. The architecture is **big-endian**, with bit 0 as the most significant bit and bit 31 as the least significant bit in a 32-bit register.

Logic Technology

- **Custom DTL (Diode-Transistor Logic)**
- Multi-board design (no single-chip CPU)
- Logic gates, flip-flops, and small combinational functions implemented in SSI/MSI style

---

Control Unit

The hardwired control unit sequences instructions through a fixed timeline of clock phases:

- **Up to 4 Prep Phases** — instruction decode and effective address calculation
- **Up to 16 Execution Phases** — instruction execution
- Each phase corresponds to one clock cycle
- **Instruction fetch is overlapped with the last phase of the previous instruction**, keeping the memory bus productive and acting as a simple form of pipelining
- Simpler instructions consume fewer phases in each group

Memory Interface

- **32-bit memory bus**, capable of accessing 8-bit (byte), 16-bit (halfword), and 32-bit (word) values
- **Single cycle memory access**
- Memory is **byte-addressable**
- Word accesses are always 4-byte aligned

- **Big-endian** byte ordering

Instruction Format

Instructions are a fixed **32 bits wide**:

```
| I (1b) | Opcode (7b) | R (4b) | X (3b) | Address (17b) |
```

- **I** — indirect addressing bit
- **Opcode** — 7 bits (128 possible opcodes)
- **R** — primary register operand (4 bits, selecting one of 16 registers)
- **X** — index register (3 bits, selecting one of 8 registers; 0 = no indexing)
- **Address** — 17-bit base address

Register File

- **16 general-purpose 32-bit user-visible registers (R0–R15)**
- Implemented as a **flip-flop array**
- Mapped to **word addresses 0–15** in the memory address space
- Register reads and writes use the same datapath as memory operations:
  - Reads: register contents loaded into **A** or **C**
  - Writes: **S (sum bus)** routes result back as a "memory write" to address 0–15
- Register pairs (e.g., R0+R1) used for double-precision floating point and 64-bit operations

---

Internal CPU Registers

Register	Width	Role
A	32 bits	Primary ALU input; source for condition code testing
B	32 bits	Multiply/divide pair with A; forms 64-bit A:B pair
C	32 bits	Memory interface register; transparent latch
D	32 bits	Secondary ALU input (fed primarily from C)

Register	Width	Role
E	8 bits	Floating-point exponent register
O	7 bits	Current opcode register
P	19 bits (bits 15–33)	Effective byte address register (byte address counter)
Q	17 bits (bits 15–31)	Next instruction address register
CC	4 bits (bits 1–4)	Condition code register

Notes on Key Registers

C Register — Transparent Latch: C operates as a **transparent latch**, meaning its output propagates while it is being loaded. This is a critical timing feature enabling single-cycle data movement:

- Memory → C → O (instruction fetch and decode in one cycle)
- Memory → C → D (memory operand available to ALU in one cycle)
- Memory[C] → C (indirect address resolution in one cycle, since C drives the address while simultaneously being overwritten with the result)

A Register:

- Loaded from the user register file or from S
- Primary source for condition code testing at end of instruction

D Register:

- Generally loaded from C
- Can also be loaded from CC (4-bit condition codes) or external DIO (Direct I/O)
- S does not feed back to D

E Register:

- Can be loaded from B, S, or CC
- Used for floating-point exponent arithmetic in coordination with the integer ALU

ALU (Adder)

- A 32-bit **"adder"** functioning as a full ALU
- **Inputs:** A register and D register
- **Output:** The **S (sum) bus** — a purely combinational bus with no dedicated S register
- **Operations supported:**
  - Add
  - AND
  - OR
  - XOR (exclusive OR)
  - Invert
  - 1-bit left/right shift
  - 4-bit left/right shift (particularly useful for hexadecimal floating-point normalization)
  - Upward align byte
  - Upward align halfword

S Bus Destinations: S can route to: A, B, C, P, user-visible registers (addresses 0–15), or memory

Sub-word Store Mechanism

Sub-word stores (byte and halfword) use the upward align operations in conjunction with **memory write mask bits** that select which bytes or halfwords are actually written to the target word address:

- **Upward align byte:** replicates the least significant byte (bits 24–31) into all four byte positions of the word, producing four identical bytes. The write mask then selects which byte position(s) are written to memory.
- **Upward align halfword:** replicates the least significant halfword (bits 16–31) into both halfword positions of the word. The write mask then selects which halfword is written to memory.

This allows STB and STH to use the standard 32-bit word memory bus without requiring separate narrow write paths.

Address Registers: P and Q

P Register (19 bits)

- Holds the **byte effective address** at the end of prep phases
- 19 bits provides a 512KB byte address space (2^19)
- Used as a working register during effective address calculation

Q Register (17 bits)

- Holds the **next instruction address**
- 17 bits (instructions are always word-aligned, so the bottom 2 bits are always zero and not stored)

P/Q Sequencing Per Instruction Cycle

1. **ENDE (last execute phase):** Q used to fetch next instruction into C; O ← C[1:7]; D ← C; R ← C[8:11]; P incremented by 4; A and P[32:33] set up for EA calculation
2. **PREP1:** P[15:31] → Q (next instruction address saved to Q, freeing P for EA calculation)
3. **PREP2 (if indirect):** C ← M[C[15:31]]; D ← C (hardware masks C to bits 15–31 for address)
4. **PREP3:** P[15:31] ← A + D[15:31]; P[32:33] retains byte offset from ENDE
5. **Execute:** P holds complete effective byte address; Q holds next instruction address
6. **EX(n-1):** P[15:31] ← Q (restore next instruction address to P)
7. **EX(n):** P ← P + 4; ENDE fires, overlapping fetch of next instruction

---

Effective Address Calculation (Prep Phases)

Indirect addressing is resolved **before** indexing. The instruction word is already held in D after ENDE, and the hardware masks to bits 15–31 when using C or D as an address:

1. **PREP1:** Q ← P[15:31]; if i=0 goto PREP3
2. **PREP2 (indirect only):** C ← M[C[15:31]]; D ← C — hardware masks C to bits 15–31 for the indirect fetch; D receives the resolved pointer via transparent C latch
3. **PREP3:** P[15:31] ← S where S ← A + D[15:31]; P[32:33] unchanged from ENDE

After PREP3, P holds the complete effective byte address:

- **P[15:31]:** word address from EA calculation (A + address field)

- **P[32:33]:** byte offset within word, set during ENDE based on operand granularity

Indirect + indexing fits within the 4-phase prep budget without timing concerns.

Condition Codes (CC)

The **4-bit CC register** captures:

Bit	Condition	Detection
Negative	A[0] = 1	MSB of A register
Zero	All bits 0	NOR of all 32 bits of A
Overflow	Carry/overflow from adder	Captured directly from ALU
Carry	Carry out of adder	Captured directly from ALU

- CC is **set by testing A** at the end of an instruction (not S directly)
- This implies S → A is the penultimate execution step, with CC testing as the final step
- CC can be loaded into D or E for arithmetic use

Multiply and Divide

- **A and B form a 64-bit register pair** that shifts together during multiply/divide
- **Multiplication:** Bit-pair (Booth-style) algorithm; 32-bit multiply takes **16 execution cycles** plus a few cycles for setup and sign adjustment
- **Division:** Non-restoring division algorithm, also using the A:B pair
- During multiply/divide loops, S feeds back into A each cycle without touching memory; B shifts in concert with A

Floating Point

- Floating point uses the **same integer ALU datapath** — no separate FP hardware
- Uses **hexadecimal (base-16) floating point** format (similar to IBM System/360):
  - **Single precision (32-bit):** 1 sign bit, 7-bit hex exponent (excess-64), 24-bit mantissa

    - **Double precision (64-bit):** 1 sign bit, 7-bit exponent, 56-bit mantissa; uses A:B register pair
- The **E register** handles exponent arithmetic in parallel with mantissa operations in A/B/D/S
- **Normalization** uses the 4-bit shift capability of the ALU (one shift = one hex digit = one exponent adjustment in E)
- Sign is held in A[0] (MSB)
- Condition codes are set by testing A at the end of floating-point instructions

---

Instruction Execution Lifecycle

The complete instruction execution lifecycle is best understood through the **ENDE signal** and the prep/execute phase structure. ENDE fires at the last execute phase of every instruction and initiates the fetch of the next instruction, overlapping it with the end of the current instruction.

ENDE Signal

ENDE performs the following simultaneously in the last execute phase:

```
ENDE: C ← M[Q]       ; fetch next instruction word from address in Q
      O ← C[1:7]     ; load opcode (bits 1-7) into O register
      D ← C           ; instruction word also loaded into D via transparent C
latch
      R ← C[8:11]     ; load R field (bits 8-11) into 4-bit R register
      P ← P + 4       ; increment P to next instruction byte address

      ; Set up A and P[32:33] for effective address calculation:
      if x=0:             A ← 0;            P[32:33] ← 00
      if x≠0, byte:       A ← RR[x] >> 2;  P[32:33] ← byte_offset
      if x≠0, halfword:   A ← RR[x] >> 1;  P[32:33] ← halfword_offset
      if x≠0, word:       A ← RR[x];       P[32:33] ← 00
      if x≠0, doubleword: A ← RR[x] << 1;  P[32:33] ← 00
```

The **transparent C latch** allows O, D, and R to all be loaded from the instruction word in the same cycle as the memory fetch.

Prep Phases

The prep phases calculate the effective address (EA) using the instruction word already held in D, and save the next instruction address to Q:

```
PREP1: Q ← P[15:31]           ; save next instruction address to Q, freeing P for
EA calc
       if i=0: goto PREP3      ; skip indirect resolution if not indirect

PREP2: C ← M[C[15:31]]; D ← C ; indirect: fetch word at address field (hardware
masks C to bits 15-31)

PREP3: P[15:31] ← S            ; S ← A + D[15:31]; word EA into P[15:31]
                                ; P[32:33] retains byte offset set during ENDE
```

After PREP3, P holds the complete effective byte address:

- **P[15:31]:** word address from EA calculation
- **P[32:33]:** byte offset within word, set during ENDE

Example: LW (Load Word, opcode 0x32)

```
; ENDE of previous instruction:
ENDE: C ← M[Q]; O ← C[1:7]; D ← C; R ← C[8:11]
      P ← P + 4
      if x=0:             A ← 0;            P[32:33] ← 00
      if x≠0, byte:       A ← RR[x] >> 2;  P[32:33] ← byte_offset
      if x≠0, halfword:   A ← RR[x] >> 1;  P[32:33] ← halfword_offset
      if x≠0, word:       A ← RR[x];       P[32:33] ← 00
      if x≠0, doubleword: A ← RR[x] << 1;  P[32:33] ← 00

; Prep phases:
PREP1: Q ← P[15:31]; if i=0 goto PREP3
PREP2: C ← M[C[15:31]]; D ← C
PREP3: P[15:31] ← S where S ← A + D[15:31]

; Execute phases:
EX1: C ← M[P]; A ← C                    ; load word from EA via transparent C
latch
EX2: S ← A; RR[r] ← S; P[15:31] ← Q    ; write to destination register, restore P
from Q
EX3: CC ← test(A); ENDE                  ; set CC, fetch next instruction
```

Unit Test Cases for LW

Case	i	x	Instruction addr field	RR[x]	EA
Direct, non-indexed	0	0	0x0100	—	0x0100
Direct, word-indexed	0	3	0x0100	5	0x0105
Indirect, non-indexed	1	0	0x0100 (→ 0x5000)	—	0x5000
Indirect, word-indexed	1	3	0x0100 (→ 0x5000)	5	0x5005

Supervisor / User Mode

The CPU has two privilege levels. Privileged instructions (I/O, PSW manipulation, memory map changes) can only execute in **supervisor mode**. A trap or interrupt automatically switches to supervisor mode and saves the user PSW.

- **CAL1–CAL4** are user-mode instructions providing a structured mechanism for user programs to request system services — not privileged themselves but transfer control to supervisor-mode handlers
- **EXU** (Execute) and **INT** (Interpret) are likewise unprivileged user instructions
- All **I/O instructions** (SIO, TIO, TDV, HIO, RD, WD, AIO) are privileged and may only execute in supervisor mode

---

I/O Architecture

- I/O has its **own separate datapath to memory** (does not contend with CPU for the memory bus)
- Channel-based I/O model: SIO (Start I/O) initiates autonomous channel programs
- Completion signaled via interrupt

Instruction Set

Instructions are listed in opcode order (hex 00–7F). Opcodes marked **NAO** (Non-Allowed Operation) trigger a non-allowed operation trap if executed.

Opcode Table

Opcode (hex)	Mnemonic	Description
00	NAO00	Non-Allowed Operation
01	NAO01	Non-Allowed Operation
02	LCFI	Load Conditions and Floating Control Immediate
03	NAO03	Non-Allowed Operation
04	CAL1	Call 1
05	CAL2	Call 2
06	CAL3	Call 3
07	CAL4	Call 4
08	PLW	Pull Word
09	PSW	Push Word
0A	PLM	Pull Multiple
0B	PSM	Push Multiple
0C	NAO0C	Non-Allowed Operation
0D	NAO0D	Non-Allowed Operation
0E	LPSD	Load Program Status Doubleword
0F	XPSD	Exchange Program Status Doubleword
10	AD	Add Doubleword
11	CD	Compare Doubleword
12	LD	Load Doubleword
13	MSP	Modify Stack Pointer
14	NAO14	Non-Allowed Operation
15	STD	Store Doubleword

Opcode (hex)	Mnemonic	Description
16	NAO16	Non-Allowed Operation
17	NAO17	Non-Allowed Operation
18	SD	Subtract Doubleword
19	CLM	Compare with Limits in Memory
1A	LCD	Load Complemented Doubleword
1B	LAD	Load Absolute Doubleword
1C	FSL	Floating Subtract Long
1D	FAL	Floating Add Long
1E	FDL	Floating Divide Long
1F	FML	Floating Multiply Long
20	AI	Add Immediate
21	CI	Compare Immediate
22	LI	Load Immediate
23	MI	Multiply Immediate
24	SF	Shift Floating
25	S	Shift
26	NAO26	Non-Allowed Operation
27	NAO27	Non-Allowed Operation
28	CVS	Convert by Subtraction
29	CVA	Convert by Addition
2A	LM	Load Multiple
2B	STM	Store Multiple
2C	NAO2C	Non-Allowed Operation

Opcode (hex)	Mnemonic	Description
2D	NAO2D	Non-Allowed Operation
2E	WAIT	Wait
2F	LRP	Load Register Pointer
30	AW	Add Word
31	CW	Compare Word
32	LW	Load Word
33	MTW	Modify and Test Word
34	NAO34	Non-Allowed Operation
35	STW	Store Word
36	DW	Divide Word
37	MW	Multiply Word
38	SW	Subtract Word
39	CLR	Compare with Limits in Register
3A	LCW	Load Complemented Word
3B	LAW	Load Absolute Word
3C	FSS	Floating Subtract Short
3D	FAS	Floating Add Short
3E	FDS	Floating Divide Short
3F	FMS	Floating Multiply Short
40	TTBS	Translate and Test Byte String
41	TBS	Translate Byte String
42	NAO42	Non-Allowed Operation
43	NAO43	Non-Allowed Operation

Opcode (hex)	Mnemonic	Description
44	ANLZ	Analyze
45	CS	Compare Selective
46	XW	Exchange Word
47	STS	Store Selective
48	EOR	Exclusive OR
49	OR	OR
4A	LS	Load Selective
4B	AND	AND
4C	SIO	Start I/O
4D	TIO	Test I/O
4E	TDV	Test Device
4F	HIO	Halt I/O
50	AH	Add Halfword
51	CH	Compare Halfword
52	LH	Load Halfword
53	MTH	Modify and Test Halfword
54	NAO54	Non-Allowed Operation
55	STH	Store Halfword
56	DH	Divide Halfword
57	MH	Multiply Halfword
58	SH	Subtract Halfword
59	NAO59	Non-Allowed Operation
5A	LCH	Load Complemented Halfword

Opcode (hex)	Mnemonic	Description
5B	LAH	Load Absolute Halfword
5C	NAO5C	Non-Allowed Operation
5D	NAO5D	Non-Allowed Operation
5E	NAO5E	Non-Allowed Operation
5F	NAO5F	Non-Allowed Operation
60	CBS	Compare Byte String
61	MBS	Move Byte String
62	NAO62	Non-Allowed Operation
63	EBS	Edit Byte String
64	BDR	Branch on Decrementing Register
65	BIR	Branch on Incrementing Register
66	AWM	Add Word to Memory
67	EXU	Execute
68	BCR	Branch on Condition Reset
69	BCS	Branch on Condition Set
6A	BAL	Branch and Link
6B	INT	Interpret
6C	RD	Read Direct
6D	WD	Write Direct
6E	AIO	Acknowledge I/O Interrupt
6F	MMC	Move to Memory Control
70	LCF	Load Control and Floating
71	CB	Compare Byte

Opcode (hex)	Mnemonic	Description
72	LB	Load Byte
73	MTB	Modify and Test Byte
74	STCF	Store Condition and Floating Control
75	STB	Store Byte
76	PACK	Pack Decimal Digits
77	UNPK	Unpack Decimal Digits
78	DS	Decimal Subtract
79	DA	Decimal Add
7A	DD	Decimal Divide
7B	DM	Decimal Multiply
7C	DSA	Decimal Shift Arithmetic
7D	DC	Decimal Compare
7E	DL	Decimal Load
7F	DST	Decimal Store

Functional Groups

Word Arithmetic: AW, SW, MW, DW, AI, CI, LI, MI

Halfword Arithmetic: AH, SH, MH, DH

Doubleword Arithmetic: AD, SD, CD, LD, STD, LCD, LAD

Word Logical: AND, OR, EOR

Load/Store Word: LW, STW, LCW, LAW

Load/Store Halfword: LH, STH, LCH, LAH

- **LH** sign-extends the 16-bit halfword to 32 bits (bit 16 replicated into bits 0–15 of destination register)

Load/Store Byte: LB, STB, CB

- **LB** zero-extends; upper 24 bits of destination register set to 0

Selective (Masked) Operations: LS, STS, CS, CLR, CLM

Modify and Test: MTW, MTH, MTB

Load/Store Multiple: LM, STM, PLW, PSW, PLM, PSM

Shift Integer: S

Shift Floating: SF

Floating Point Short: FAS, FSS, FMS, FDS

Floating Point Long (Doubleword): FAL, FSL, FML, FDL

Decimal: DA, DS, DM, DD, DSA, DC, DL, DST, PACK, UNPK

Branch: BCR, BCS, BAL, BDR, BIR

String Operations: CBS, MBS, EBS, TTBS, TBS

Byte/Field Manipulation: ANLZ, LCF, LCFI, STCF, XW, AWM, CVS, CVA

Stack Operations: MSP, PLW, PSW, PLM, PSM

System Calls (user mode): CAL1, CAL2, CAL3, CAL4

Other User Instructions: EXU, INT

System/Privileged: LPSD, XPSD, WAIT, LRP, MMC

I/O (privileged): SIO, TIO, TDV, HIO, RD, WD, AIO

Design Philosophy

The Sigma 7 CPU design reflects consistent optimization for capability within DTL transistor budget constraints:

- **Transparent C latch** eliminates pipeline registers, squeezing maximum work per clock cycle
- **Register-file-as-memory mapping** eliminates a separate register read path, reusing the existing memory datapath
- **P/Q address register dance** implements instruction sequencing with minimal hardware
- **Shared integer/FP ALU** with E register augmentation avoids duplicating the expensive 32-bit datapath

- **Hardwired control** with fixed phase counts provides predictable timing without microcode store overhead
- **S bus (no S register)** avoids an extra register while still allowing flexible result routing

The result is a machine that delivers 32-bit capability, hardware multiply/divide, floating point, and virtual memory support in a lean, carefully optimized DTL implementation.